



Redis: Data Sharing & Pub/Sub

Sistemas Distribuídos (XRSC09) - Unifei

Professor: Rafael Frinhani

Integrantes: Gabriel Belluco, Pedro Furlaneto, Melissa Arantes, Davi Rodrigues

08/05/2026

| SUMÁRIO

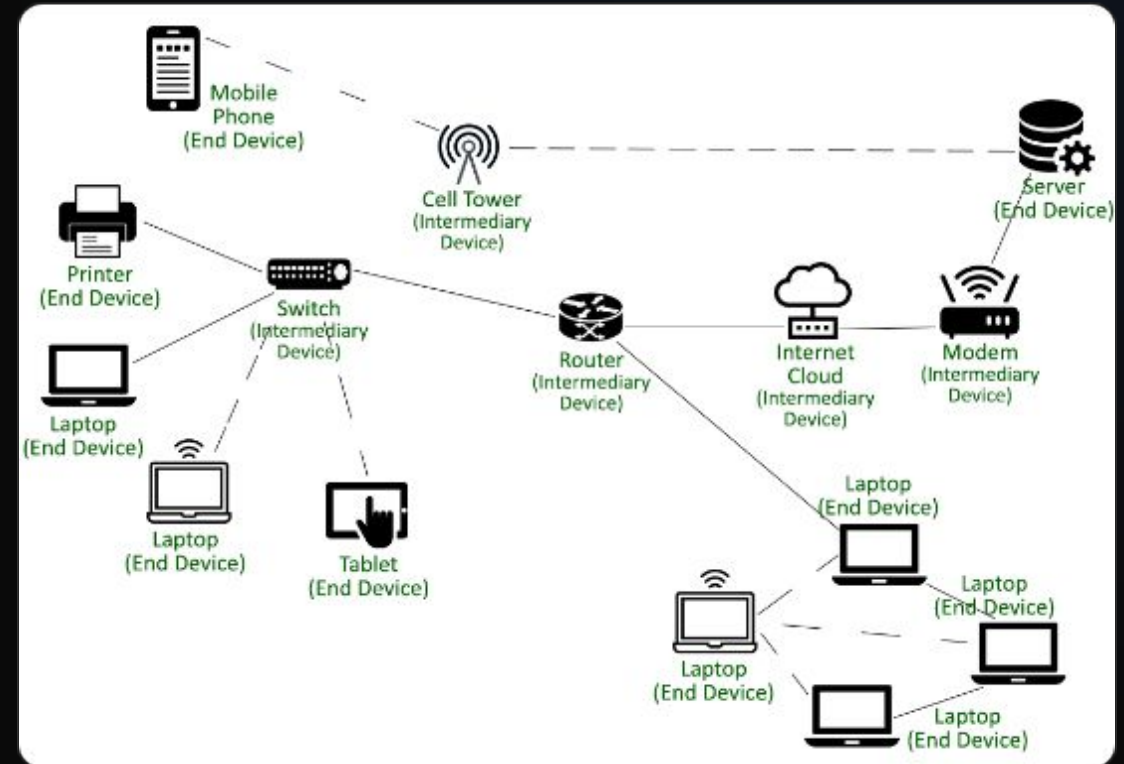
- ✓ Contexto: O Problema da Comunicação
- ✓ Introdução ao Redis
- ✓ Mecanismo de Pub/Sub
- ✓ Compartilhamento de Estado (Data Sharing)
- ✓ Arquitetura do Protótipo
- ✓ Demonstração Prática
- ✓ Avaliação Crítica e Limitações
- ✓ Conclusão e Perguntas

O DESAFIO DISTRIBUÍDO

Como conectar processos?

Em sistemas distribuídos, o acoplamento direto entre cliente e servidor gera gargalos.

- ✓ **Indisponibilidade:** O que ocorre se o receptor estiver offline?
- ✓ **Estado:** Como garantir que múltiplos processos enxerguem o mesmo dado?
- ✓ **Escalabilidade:** Como adicionar workers sem alterar o cliente?



REDIS COMO MIDDLEWARE



In-Memory Data Store

Armazenamento primário em RAM, garantindo latências de sub-milissegundos para operações complexas.



Estruturas de Dados

Mais que um cache: suporta Hashes, Lists, Sets e Strings de forma atômica e eficiente.



Broker Central

Atua como o "correio" do sistema, desacoplando quem envia (Publisher) de quem processa (Subscriber).

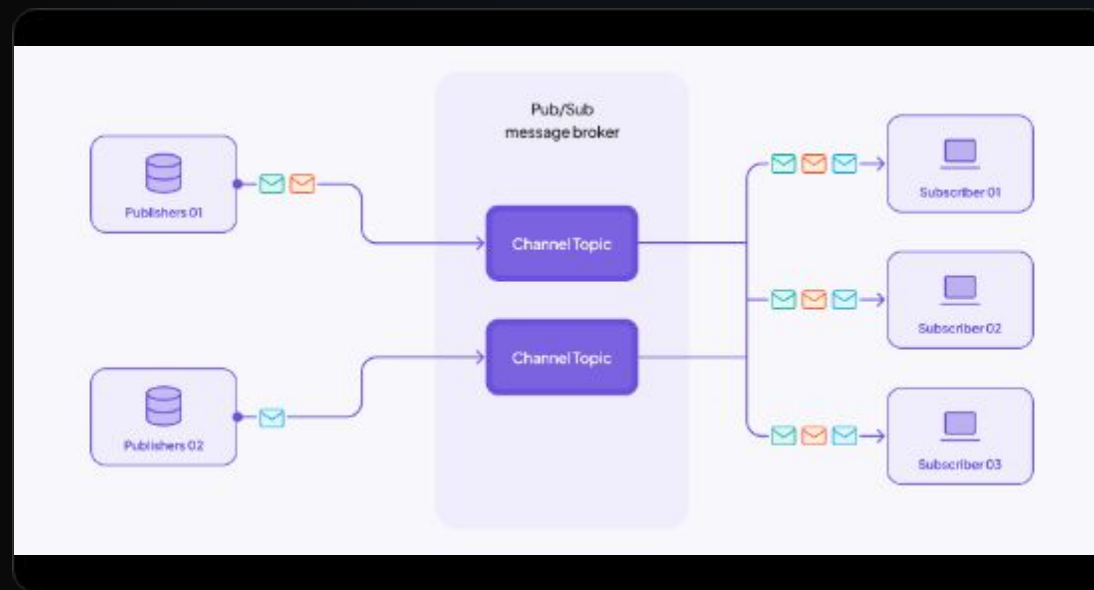
MECANISMO PUB/SUB

O modelo **Publisher/Subscriber** é assíncrono e baseado em canais (Channels).

Canais do Projeto

- redis_requests (Req JSON)
- redis_responses (Res JSON)

Fire-and-forget: O Redis entrega a mensagem para quem estiver ouvindo no momento exato.

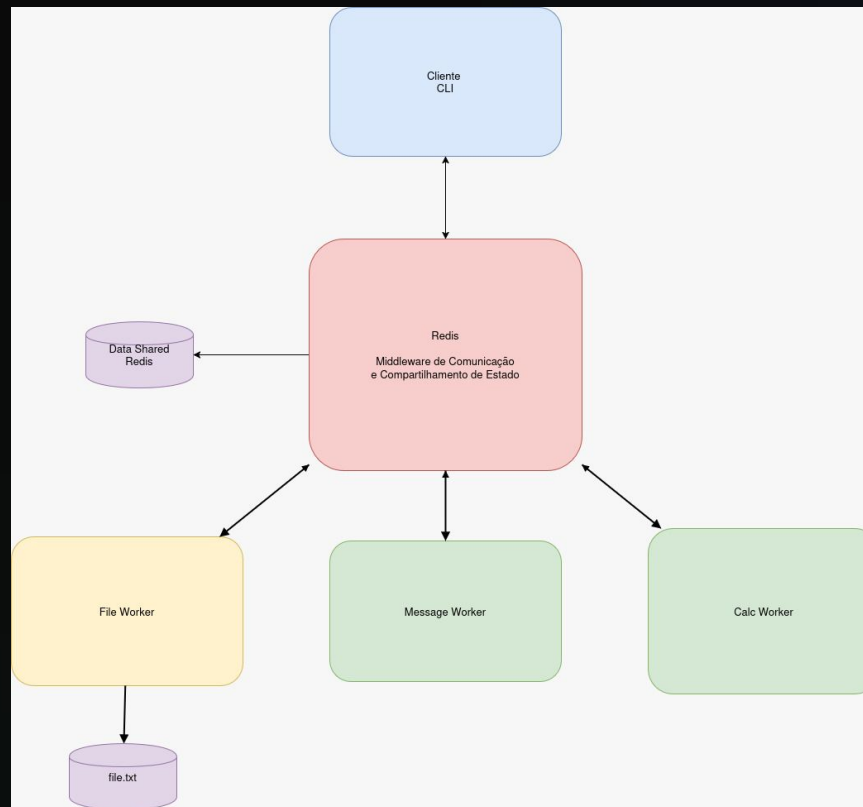


DATA SHARING: ESTADO GLOBAL

Diferente do Pub/Sub, o **Data Sharing** mantém a persistência do estado atual para consulta posterior por qualquer nó.

Chave Redis	Tipo	Propósito
resultado:ID	Hash	Dados da operação finalizada
request_history	List	Log cronológico de chamadas
calc_requests	String	Contador atômico (Métricas)

ARQUITETURA DO PROTÓTIPO



CLIENTE (Python)
Publisher/Subscriber



REDIS BROKER
Memory + Pub/Sub



WORKERS (N)
Distributed Processing

FLUXO DE OPERAÇÃO

- ✓ Cliente gera request_id único.
- ✓ Publica JSON em redis_requests.
- ✓ Workers recebem; apenas o responsável processa.
- ✓ Worker salva estado em resultado:id.
- ✓ Worker publica "OK" em redis_responses.
- ✓ Cliente lê aviso e busca dado no Hash.

```
file_worker-1
{
  "request_id": "c7788f89-cdf0-4a2e-8de1-66b0d5b506c7",
  "operation": file
  "content": "Alterando arquivo 1"
}
```


WORKERS ESPECIALIZADOS

Calc Worker

Processamento intensivo. Realiza cálculos e incrementa contadores globais.

Message Worker

Tratamento de texto e persistência de histórico de chat na `request_history`.

File Worker

Interface com o SO. Altera arquivos texto e sincroniza metadados no Redis.

"O desacoplamento permite que cada worker escale independentemente conforme a carga."

DEMONSTRAÇÃO PRÁTICA

```
PS C:\Users\flavi\Documents\Descubra\Pessoal\Redis\Redis-XRSC09> docker-compose up --build
cept connections from any IP address on any network interface.

calc_worker-1 | calc_worker conectado em redis:6379.
message_worker-1 | message_worker conectado em redis:6379.
calc_worker-1 | Aguardando mensagens no canal 'redis_requests'...
message_worker-1 | Aguardando mensagens no canal 'redis_requests'...
file_worker-1 | Aguardando mensagens no canal 'redis_requests'...
message_worker-1 | message_worker recebido: {'operation': 'message', 'content': 'Enviando mensagem de texto 1', 'request_id': 'cf255d55-b47a-4c24c41-e9b5e5492944'}
file_worker-1 | file_worker recebido: {'operation': 'file', 'content': 'Alterando arquivo 1', 'request_id': 'c7788f89-cdf0-4a2e-8de1-66b0d5b507'}
file_worker-1 | file_worker recebido: {'operation': 'file', 'content': 'alterando arquivo 2', 'request_id': '46b3e4b9-a8da-4880-9d0a-7058e8aa6c'}
file_worker-1 | file_worker recebido: {'operation': 'file', 'content': 'alterando arquivo 3', 'request_id': '5b2f2807-3b6f-4586-b695-5ca650b0df'}
calc_worker-1 | calc_worker recebido: {'operation': 'calc', 'a': 231321.0, 'b': 0.5, 'operator': '**', 'request_id': '38ceed93-7413-4f08-970e-ta473617e5'}
message_worker-1 | message_worker recebido: {'operation': 'history', 'request_id': 'db02936e-5f42-41bf-a40b-fdc8252bf989'}
[]
```

Logs de Execução

À esquerda, vemos os workers processando requisições em tempo real.

- ✓ Captura via Pub/Sub
- ✓ Validação de operation
- ✓ Escrita em memória
- ✓ Callback de resposta

PERSISTÊNCIA

Pub/Sub

Se o subscriber estiver offline, **a mensagem é perdida**.

Ideal para notificações em tempo real onde o "agora" importa mais que o histórico.

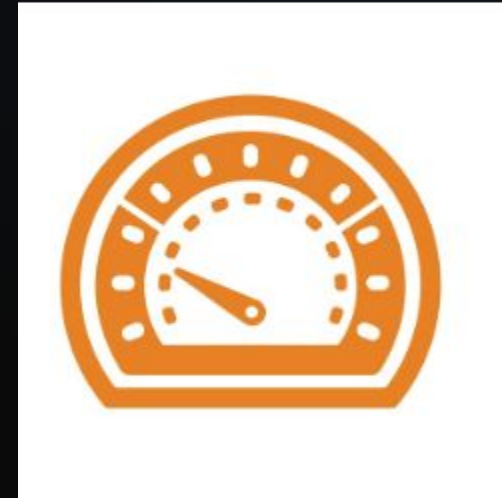
Data Sharing

O resultado salvo em resultado:id persiste conforme a configuração (RDB/AOF). O cliente pode buscar o dado mesmo após o worker terminar.

Ponto Chave: Resolvemos a limitação do Pub/Sub usando Hashes para "estacionar" o resultado processado.

POR QUE O REDIS É RÁPIDO?

- ✓ **In-Memory:** Sem I/O de disco para leitura/escrita de comandos.
- ✓ **Single-Threaded Event Loop:** Evita context switching e race conditions.
- ✓ **I/O Multiplexing:** Gerencia milhares de conexões simultâneas via epoll.



| CONCLUSÃO

O projeto demonstrou como o Redis atua como o sistema nervoso central de uma aplicação distribuída.

- ✓ Desacoplamento total entre Cliente e Workers.
- ✓ Comunicação assíncrona de baixa latência.
- ✓ Compartilhamento de estado global e consistente.

Perguntas?

Dúvidas sobre Pub/Sub, Persistência ou Arquitetura?